

C++&Qt 알고리즘 보고서(Day3)

학번: 2026406076

이름: 정창규

제출일: 2026-09-07

목차

1. 알고리즘이란?
 - 1.1 알고리즘의 개념
 - 1.2 알고리즘의 조건
2. 알고리즘의 효율성
 - 2.1 시간 복잡도
 - 2.2 공간 복잡도
 - 2.3 시간 복잡도의 종류
3. 그래프와 최단 경로 탐색
 - 3.1 그래프의 개념
 - 3.2 정점, 간선, 가중치
 - 3.3 최단 경로 탐색의 개념
4. Dijkstra 알고리즘
 - 4.1 Dijkstra 알고리즘의 개념
 - 4.2 Dijkstra 알고리즘의 동작 원리
 - 4.3 누적 비용을 이용한 탐색
 - 4.4 Dijkstra 알고리즘의 탐색 과정
 - 4.5 Dijkstra 알고리즘의 특징
5. A* 알고리즘
 - 5.1 A* 알고리즘의 개념
 - 5.2 $g(n)$, $h(n)$, $f(n)$
 - 5.3 A* 알고리즘의 동작 원리
 - 5.4 A* 알고리즘의 탐색 과정
 - 5.5 A* 알고리즘의 특징
6. Dijkstra 알고리즘과 A* 알고리즘 비교
 - 6.1 탐색 방식 비교
 - 6.2 최종 경로 비용 비교
 - 6.3 탐색한 노드 수 비교
 - 6.4 실행 시간 비교
7. 정리 및 느낀 점

1. 알고리즘이란?

1.1 알고리즘의 개념

- 특정한 문제나 작업을 해결하기 위해 정해진 순서에 따라 수행하는 절차
- 입력된 값을 일정한 과정으로 처리하여 원하는 결과를 얻기 위한 계산 과정
- 알고리즘 작성 전 해결해야 할 작업의 명확한 정의 필요
- 문제 해결에 필요한 입력값과 최종 출력값의 구체적인 설정
- 기본적인 처리 흐름
 - 입력(Input) → 처리(Process) → 출력(Output)

1.2 알고리즘의 조건

- 각 단계에서 수행해야 할 작업의 명확한 정의 필요
- 명확성
 - 각 단계가 모호하지 않도록 구성
 - 수행해야 할 작업을 정확하게 판단할 수 있도록 작성
- 유한성
 - 무한하게 반복되지 않고 일정한 단계 후 종료
 - 문제 해결 과정의 종료 조건 필요
- 입력, 처리, 출력뿐만 아니라 각 단계의 명확성과 종료 여부까지 고려하여 구성

2. 알고리즘의 효율성

2.1 시간 복잡도

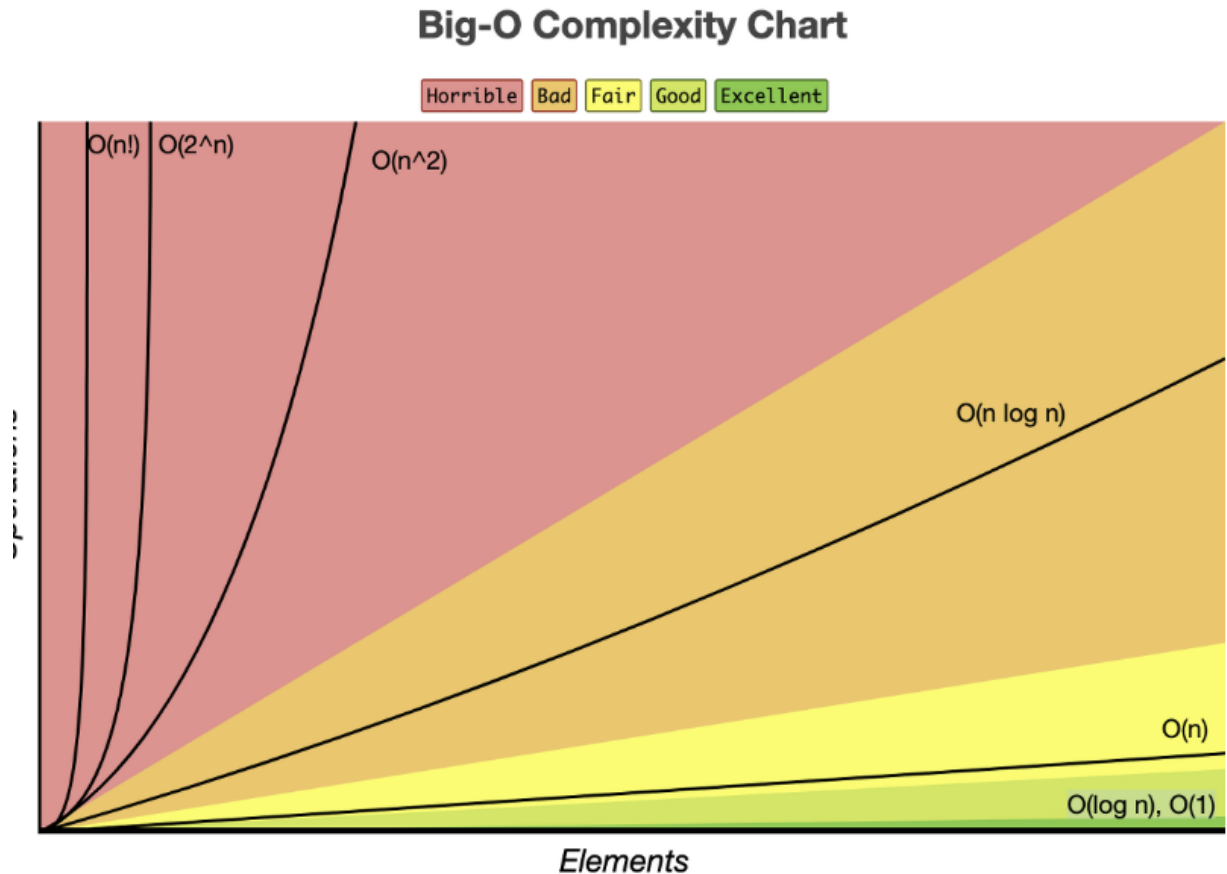
- 입력 크기 n 증가에 따른 연산 횟수의 증가 정도
- 입력 데이터가 많아질수록 알고리즘의 처리량이 어떻게 변화하는지 판단하는 기준
- 작은 입력보다 입력 크기가 충분히 커졌을 때의 변화에 중점
- 입력 크기가 증가할 때의 성능 변화를 분석하는 점근적 분석 사용

2.2 공간 복잡도

- 입력 크기 n 증가에 따른 메모리 사용량의 증가 정도
- 같은 문제를 해결하는 알고리즘이라도 서로 다른 메모리 사용량 발생 가능

- 알고리즘의 효율성을 판단할 때 수행 시간과 함께 고려할 요소
- 입력 데이터가 많아질수록 필요한 저장 공간의 변화 정도를 비교하는 기준

2.3 시간 복잡도의 종류



- **$O(1)$**
 - 입력 크기와 관계없이 일정한 연산량
 - 배열의 특정 위치에 접근하는 경우
- **$O(\log n)$**
 - 입력 크기가 증가해도 연산 횟수가 비교적 천천히 증가
 - 이진 탐색이 해당됨
- **$O(n)$**
 - 입력 크기에 비례하여 연산 횟수 증가
 - 선형 탐색이 해당됨
- **$O(n \log n)$**

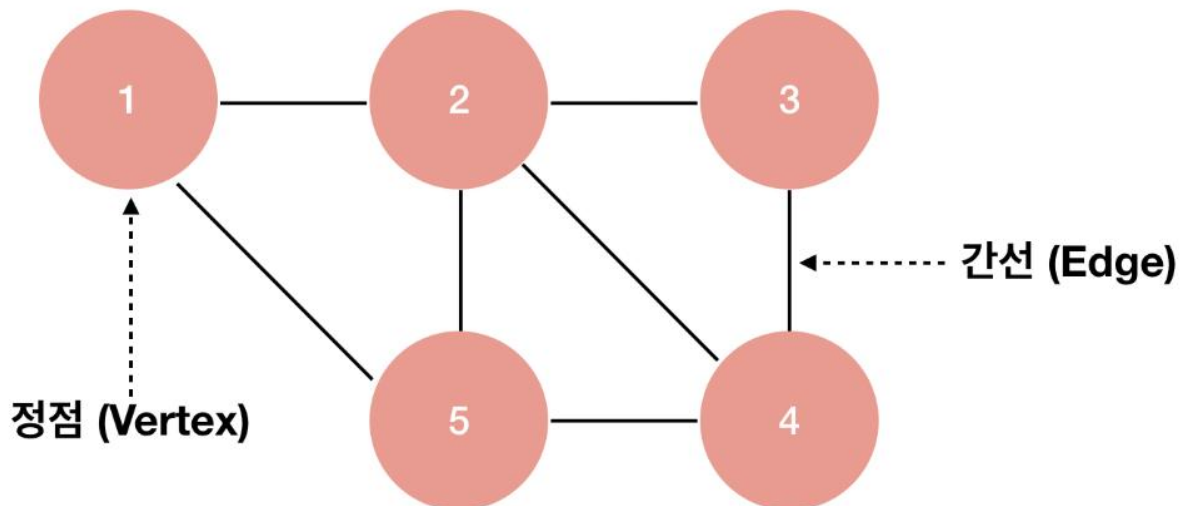
- $O(n)$ 보다 빠르게 증가하지만 $O(n^2)$ 보다는 작은 증가량
- 병합 정렬이 해당됨
- $O(n^2)$
 - 입력 크기가 증가할수록 연산 횟수가 크게 증가
 - 이중 반복문이 해당됨
- 입력 크기가 커질수록 시간 복잡도에 따른 연산량 차이가 더욱 크게 발생

3. 그래프와 최단 경로 탐색

3.1 그래프의 개념

- 여러 개의 정점(Vertex)과 정점을 연결하는 간선(Edge)으로 구성
- 정점: 하나의 위치 또는 대상
- 간선: 정점과 정점 사이의 연결 관계
- 여러 위치 사이의 이동 관계나 연결 구조를 표현하는 데 활용

3.2 정점, 간선, 가중치



- 정점(Vertex)
 - 그래프를 구성하는 하나의 지점
- 간선(Edge)
 - 두 정점을 연결하는 경로

- 가중치(Weight)
 - 간선에 부여되는 값
 - 거리, 시간, 비용, 위험도 등 다양한 기준으로 설정 가능

3.3 최단 경로 탐색의 개념

- 시작점에서 도착점까지 이동 가능한 여러 경로 중 비용이 가장 작은 경로 탐색
- 가중치가 있는 그래프에서는 단순한 정점 개수보다 전체 누적 비용을 기준으로 경로 판단
- 최단 경로 탐색에 활용되는 대표적인 알고리즘
 - Dijkstra 알고리즘
 - A* 알고리즘
- 두 알고리즘은 모두 최단 경로를 찾지만 탐색 기준과 탐색 방식에서 차이 발생

4. Dijkstra 알고리즘

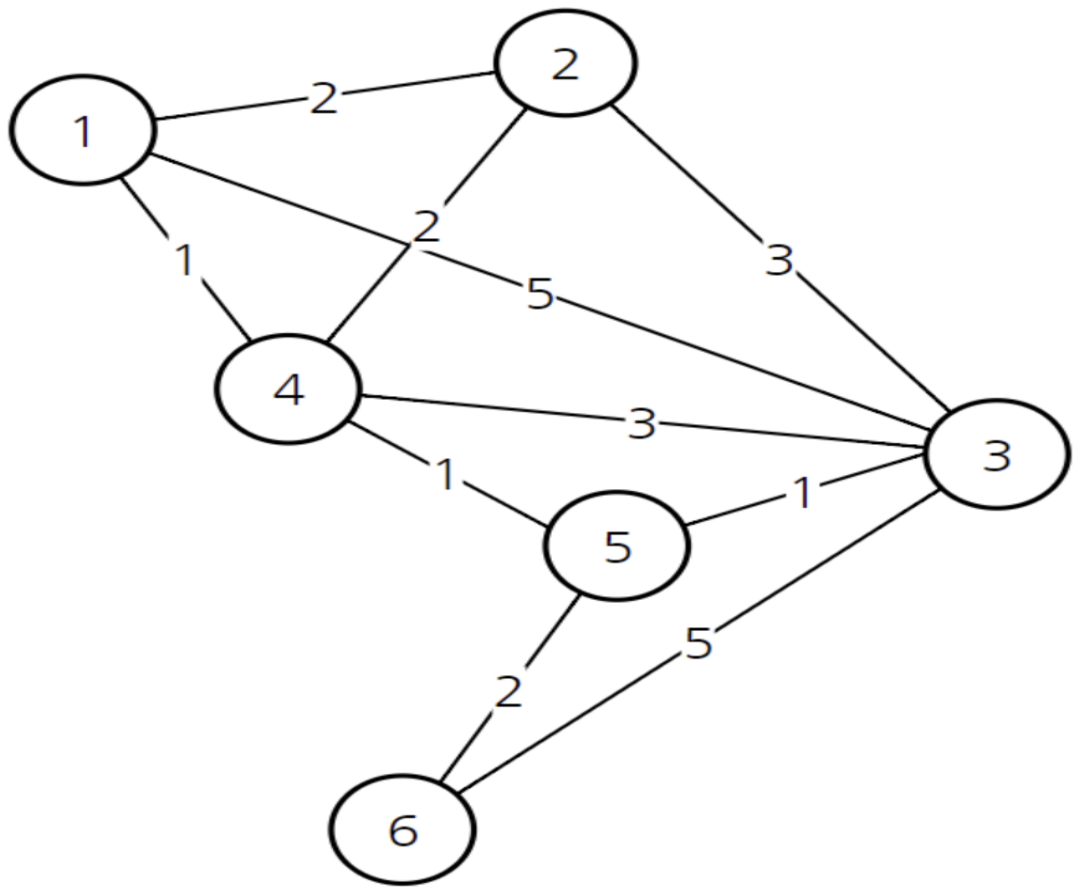
4.1 Dijkstra 알고리즘의 개념

- 그래프에서 시작점으로부터 다른 정점까지의 최단 경로 탐색
- 각 간선에 부여된 가중치를 이용하여 경로 비용 계산
- 시작점에서 현재 정점까지의 누적 비용을 기준으로 탐색
- 더 작은 누적 비용을 가지는 경로를 우선적으로 선택
- 사용되는 간선 비용은 0 이상

4.2 Dijkstra 알고리즘의 동작 원리

- 시작 정점의 비용을 0으로 설정
- 시작 정점과 직접 연결된 정점들의 이동 비용 확인
- 아직 방문하지 않은 정점 중 누적 비용이 가장 작은 정점 선택
- 선택된 정점과 연결된 주변 정점의 비용 계산
- 새롭게 계산된 비용과 기존 비용 비교
- 새로운 비용이 더 작을 경우 해당 값으로 갱신
- 같은 과정을 반복하여 목적지까지의 최단 경로 탐색

4.3 누적 비용을 이용한 탐색

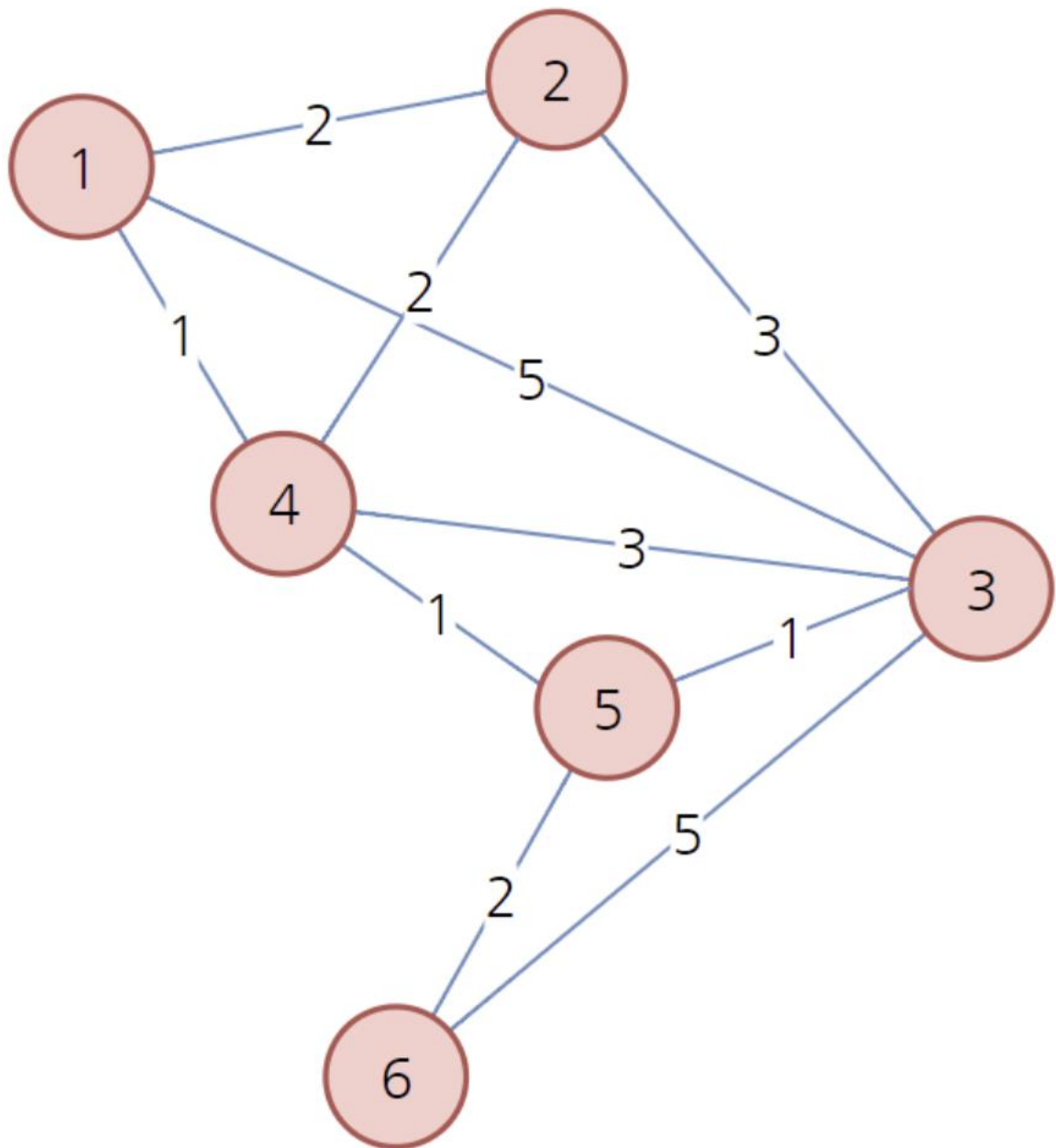


	1	2	3	4	5	6
1	0	2	5	1	∞	∞
2	2	0	3	2	∞	∞
3	5	3	0	3	1	5
4	1	2	3	0	1	∞
5	∞	∞	1	1	0	2
6	∞	∞	5	∞	2	0

- 가중치 그래프의 각 간선에 정점 사이의 이동 비용 표시
- 그래프의 연결 관계와 비용을 인접 행렬 형태로 표현 가능
- 행과 열은 각각 하나의 정점을 의미
- 두 정점이 직접 연결된 경우 해당 간선의 가중치 기록
- 직접 연결되지 않은 정점은 무한값으로 표현

- 같은 정점끼리는 비용을 0으로 표현
- 시작점에서 각 정점까지 이동하면서 발생한 비용을 누적하여 계산
- 같은 정점으로 이동하는 여러 경로가 존재할 경우 가장 작은 누적 비용 선택

4.4 Dijkstra 알고리즘의 탐색 과정



0	2	3	1	2	4
---	---	---	---	---	---

- 시작 정점을 1번으로 설정
- 1번 정점에서 각 정점까지의 최소 누적 비용

- [0, 2, 3, 1, 2, 4]
- 각 값의 의미
 - 1번 정점까지의 최소 비용: 0
 - 2번 정점까지의 최소 비용: 2
 - 3번 정점까지의 최소 비용: 3
 - 4번 정점까지의 최소 비용: 1
 - 5번 정점까지의 최소 비용: 2
 - 6번 정점까지의 최소 비용: 4
- 시작점과 연결된 정점들의 비용 확인
- 현재까지의 누적 비용이 가장 작은 정점을 다음 탐색 대상으로 선택
- 선택된 정점과 연결된 주변 정점들의 누적 비용 계산
- 기존 비용보다 더 작은 경로가 발견될 경우 값 갱신
- 누적 비용이 가장 작은 정점을 다시 선택하여 같은 과정 반복
- 모든 계산을 통해 시작점에서 각 정점까지의 최소 비용 결정

4.5 Dijkstra 알고리즘의 특징

- 시작점에서 현재 정점까지의 **실제 누적 비용** 사용
- 각 간선의 가중치를 이용한 전체 경로 비용 비교
- 간선 비용이 0 이상인 경우 사용
- 목적지까지의 예상 거리 사용하지 않음
- 현재까지 계산된 비용이 작은 노드를 중심으로 탐색
- 목적지 방향과 관계없는 노드까지 탐색 범위가 확장될 가능성 존재

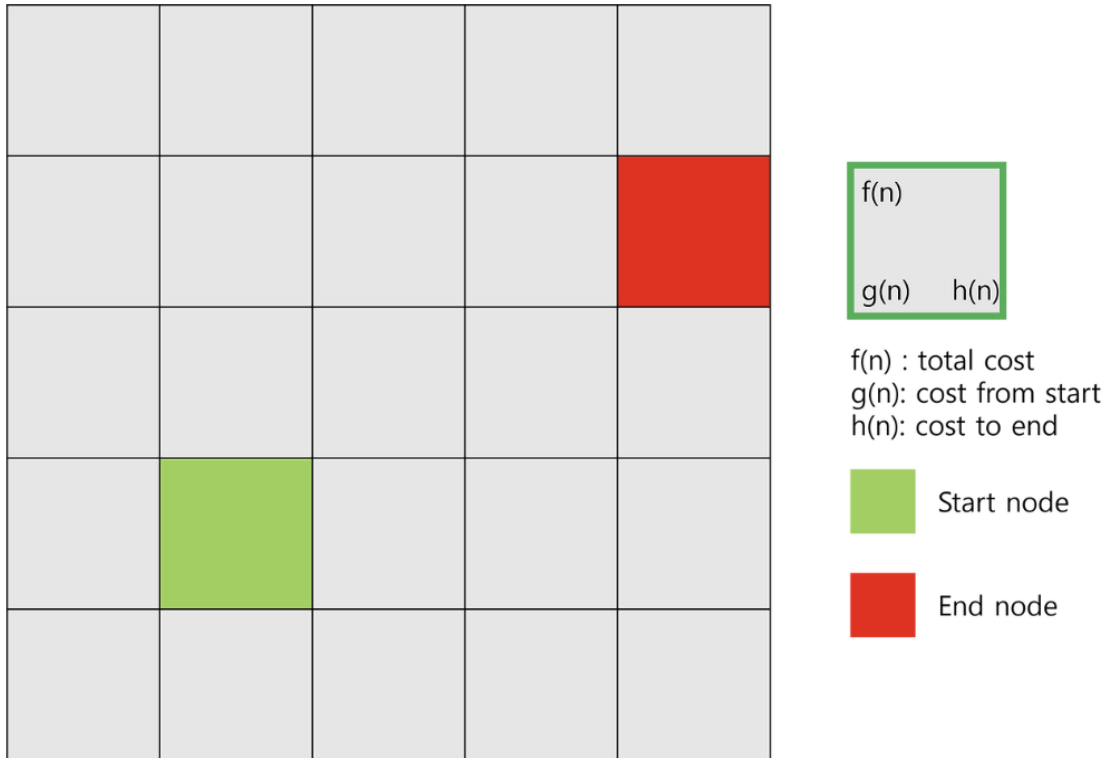
5. A* 알고리즘

5.1 A* 알고리즘의 개념

- 시작점에서 목적지까지의 최단 경로 탐색에 사용
- 시작점에서 현재 위치까지의 실제 비용과 목적지까지의 예상 비용을 함께 고려
- Dijkstra 알고리즘과 달리 목적지까지 남은 거리 또는 비용을 탐색 과정에 활용

- 목적지와 관련이 적은 방향의 탐색을 줄이고 목적지 방향으로 탐색을 집중시키는 방식

5.2 $g(n)$, $h(n)$, $f(n)$



- $g(n)$: 시작점에서 현재 노드까지 실제로 이동하는 데 사용된 비용
- $h(n)$ (휴리스틱함수) : 현재 노드에서 목적지까지 남은 비용을 예상한 값
- $f(n)$: 현재 노드의 탐색 우선순위를 판단하는 값, $g(n)$ 과 $h(n)$ 의 합
- A* 알고리즘의 비용 계산 : $f(n) = g(n) + h(n)$
- $f(n)$ 값이 작은 노드를 우선적으로 탐색

5.3 A* 알고리즘의 동작 원리

- 시작점에서 탐색 시작
- 현재 노드까지의 실제 비용 $g(n)$ 계산
- 현재 노드에서 목적지까지의 예상 비용 $h(n)$ 계산
- 두 값을 더하여 $f(n)$ 계산
- 탐색 가능한 노드 중 $f(n)$ 값이 가장 작은 노드 선택
- 선택된 노드를 기준으로 주변 노드의 비용 계산
- 목적지에 도달할 때까지 동일한 과정 반복

5.4 A* 알고리즘의 탐색 과정

	58 28 30	44 24 20	38 28 10	
64 14 50	50 10 40	44 14 30	44 24 20	
70 10 60	50 00 50	50 10 40	58 28 30	
84 14 70	70 10 60	64 14 50		

- 각 노드마다 $g(n)$, $h(n)$, $f(n)$ 값 계산
- $f(n)$ 값이 가장 작은 노드를 다음 탐색 대상으로 선택
- 선택된 노드 주변의 다른 노드에도 동일한 방식 적용
- 기존 경로보다 더 작은 비용을 가지는 경로 발견 시 값 갱신
- 목적지에 도달할 때까지 탐색 반복
- 목적지까지의 예상 비용을 함께 고려하여 탐색 방향을 목적지 쪽으로 집중

5.5 A* 알고리즘의 특징

- 실제 이동 비용 $g(n)$ 과 예상 비용 $h(n)$ 을 함께 사용
- 목적지의 위치를 고려하는 탐색 방식
- $f(n) = g(n) + h(n)$ 을 기준으로 탐색 순서 결정
- Dijkstra 알고리즘에 비해 탐색 범위를 줄일 수 있음

- 불필요한 노드 탐색 감소 가능
- $h(n)$ 설정 방식에 따라 탐색 효율 변화

6. Dijkstra 알고리즘과 A* 알고리즘 비교

6.1 탐색 방식 비교

Dijkstra 알고리즘

- 시작점에서 현재 노드까지의 실제 누적 비용 $g(n)$ 기준
- 현재까지 가장 적은 비용으로 도달할 수 있는 노드를 우선 탐색
- 목적지의 위치나 방향은 별도로 고려하지 않음

A* 알고리즘

- 실제 누적 비용 $g(n)$ 과 예상 비용 $h(n)$ 을 함께 사용
- $f(n) = g(n) + h(n)$ 을 기준으로 탐색 우선순위 결정
- 목적지의 위치를 고려하여 탐색 방향 설정

가장 큰 차이

- Dijkstra: 현재까지의 비용 중심
- A*: 현재까지의 비용 + 목적지까지의 예상 비용 중심

6.2 최종 경로 비용 비교

- 두 알고리즘 모두 비용이 가장 작은 경로 탐색을 목적으로 사용
- Dijkstra는 실제 누적 비용을 단계적으로 비교하며 최단 경로 탐색
- A*는 실제 누적 비용과 목적지까지의 예상 비용을 이용하여 최단 경로 탐색
- 탐색 과정과 방문하는 노드의 수는 서로 다를 수 있음
- 허용 가능한 휴리스틱을 사용할 경우 동일한 맵에서 두 알고리즘의 최종 경로 비용은 같은 값으로 결정됨
- 최종 경로보다 목적지에 도달하기까지 어떤 노드를 탐색했는지에서 두 알고리즘의 차이가 크게 나타남

6.3 탐색한 노드 수 비교

- Dijkstra는 목적지 방향을 고려하지 않고 누적 비용이 작은 노드를 순서대로 탐색
 - 시작점을 중심으로 여러 방향으로 탐색 범위가 넓어질 가능성 존재

- A*는 목적지까지의 예상 비용 $h(n)$ 사용
 - 목적지와 관련이 적은 방향의 탐색을 줄일 수 있음
- 동일한 조건에서 A*가 Dijkstra보다 적은 노드를 탐색할 수 있음
- 실제 탐색 노드 수는 맵의 구조, 장애물 배치, 시작점과 도착점의 위치에 따라 결정됨

6.4 실행 시간 비교

- 탐색 과정에서 확인하는 노드의 수와 계산량에 따라 실행 시간 차이 발생
- Dijkstra는 목적지 방향과 관계없는 노드까지 탐색할 수 있어 탐색량 증가 가능
- A*는 예상 비용을 추가로 계산해야 하지만 탐색 범위를 줄일 수 있다는 특징
- A*의 탐색 노드 수가 크게 줄어드는 경우 전체 실행 시간 감소 가능
- 실제 실행 시간은 맵 크기, 장애물 배치, 휴리스틱 계산 방식, 프로그램 구현 방식 등에 따라 달라짐

7. 정리 및 느낀 점

7.1 정리

- 알고리즘: 문제 해결을 위한 순서와 절차
- 알고리즘 효율성 판단 기준
 - 시간 복잡도
 - 공간 복잡도
- 그래프의 기본 구성
 - 정점
 - 간선
 - 가중치
- Dijkstra 알고리즘
 - 실제 누적 비용을 이용한 최단 경로 탐색
- A* 알고리즘
 - 실제 누적 비용과 목적지까지의 예상 비용을 함께 활용
- 두 알고리즘의 핵심적인 차이
 - Dijkstra: $g(n)$

- $A^*: g(n) + h(n)$
- 탐색 기준의 차이로 인해 탐색 노드 수와 실행 시간에서도 차이 발생 가능

7.2 느낀 점

이번 내용을 정리하면서 처음에는 Dijkstra 알고리즘과 A^* 알고리즘이 거의 비슷한 방식이라고 생각했다. 하지만 공부를 하면서 Dijkstra 알고리즘은 시작점에서 현재 노드까지의 실제 누적 비용을 기준으로 탐색하고, A^* 알고리즘은 여기에 목적지까지의 예상 비용을 추가하여 탐색한다는 차이를 알게 되었다.

특히 Dijkstra 알고리즘의 가중치 그래프를 직접 확인하면서 간선 하나의 비용과 시작점부터 계산한 누적 비용이 서로 다르다는 점을 이해할 수 있었다. 처음에는 [0, 2, 3, 1, 2, 4]의 값이 무엇을 나타내는지 헷갈렸지만, 각 값이 시작 정점에서 다른 정점까지 이동하는 최소 누적 비용이라는 것을 알게 되면서 탐색 과정도 이해하기 쉬워졌다.

A^* 알고리즘에서도 $g(n)$, $h(n)$, $f(n)$ 이 처음에는 비슷하게 느껴졌지만, 실제 비용과 예상 비용을 나누어 생각하면서 Dijkstra 알고리즘과의 차이를 이해할 수 있었다.